

Lecture 52 : Part 2: SQL Injection

Error Based

Introduction

Imagine a digital lock that, when picked incorrectly, doesn't just stay locked but shouts out clues about its internal mechanism. This is precisely the vulnerability that Error-Based SQL Injection exploits. In the complex world of web applications, seemingly innocuous error messages, often dismissed as minor glitches, can become a goldmine for attackers. These errors, triggered by malformed SQL queries, inadvertently leak critical information about the database structure, version, and even user credentials.

Understanding Error-Based SQL Injection is not merely an academic exercise; it's a fundamental skill for anyone involved in cybersecurity, whether as a defender or an ethical hacker. For students aspiring to build secure systems, recognizing these vulnerabilities is paramount to preventing catastrophic data breaches. For professors, it highlights a crucial area of practical application in database security curricula. This lecture delves into the practical techniques of exploiting such errors, using the Metasploitable 2 environment, to demonstrate how attackers can enumerate database structures and extract sensitive data, turning system failures into intelligence gains.

Core Concepts

Term	Definition	Significance
SQL Injection	A code injection technique used to attack data-driven applications, where malicious SQL statements are inserted into an entry field for execution.	It's a pervasive and dangerous web vulnerability that can lead to unauthorized data access, modification, or deletion.
Error-Based SQL Injection	A specific type of SQL Injection where an attacker relies on error messages generated by the database server to gather information	It allows attackers to enumerate database details (like column counts, versions, users) even when direct data retrieval is blocked.

Term	Definition	Significance
	about the database structure.	
ORDER BY clause	An SQL clause used to sort the result-set of a query by one or more columns.	When used with an invalid column number, it triggers an error that can reveal the actual number of columns in a table.
UNION clause	An SQL operator used to combine the result-set of two or more <code>SELECT</code> statements into a single result-set.	It's crucial for injecting malicious <code>SELECT</code> statements to retrieve data from other tables or database functions, provided column counts match.
Database Enumeration	The process of systematically discovering information about a database, such as table names, column names, data types, and database versions.	This is a critical reconnaissance step for attackers, providing the blueprint needed for further, more targeted attacks.
Metasploitable 2	A deliberately vulnerable Linux virtual machine designed for security training and testing.	Provides a safe and controlled environment to practice and understand various attack techniques, including SQL Injection, without harming real systems.

Detailed Analysis

1. Enumerating Column Count Using `ORDER BY`

One of the initial steps in an error-based SQL injection attack is to determine the number of columns in the target table. This is crucial because the `UNION` operator, which is often used for data extraction, requires that all `SELECT` statements have

the same number of columns. The `ORDER BY` clause provides an elegant way to discover this information by intentionally causing an error.

The process involves appending an `ORDER BY` clause to a legitimate SQL query and incrementally increasing the column number until the database throws an error.

- **Initial Query Setup:**

An attacker starts with a legitimate query that is expected to return an empty set or a limited result, making it easier to observe the effects of the injection. For instance, targeting an `accounts` table with non-existent credentials:

```
SELECT * FROM accounts WHERE username = 'test' AND password = 'any' ORDER BY 1;
```

This query, if `test` and `any` are not valid credentials, will return an empty set. The `ORDER BY 1` sorts by the first column, which is valid.

- **Incrementing `ORDER BY` to Induce Error:**

The attacker then systematically increases the number in the `ORDER BY` clause:

- `ORDER BY 2` -> Empty set (no error)
- `ORDER BY 3` -> Empty set (no error)
- `ORDER BY 4` -> Empty set (no error)
- `ORDER BY 5` -> Empty set (no error)
- `ORDER BY 6` -> **Error!**

The error generated by `ORDER BY 6` indicates that the table `accounts` does not have a sixth column. This immediately tells the attacker that the table has exactly **5 columns**. The database engine attempts to sort by a non-existent column, leading to a predictable error message.

- **Confirming Column Count:**

To verify this, a simple `SELECT * FROM accounts;` query (if accessible) would confirm the number of columns. In the demonstrated scenario, the `accounts` table indeed has 5 columns. This technique is highly effective because it relies on the database's internal error handling, which often reveals structural information.

2. Extracting Data Using `UNION` and Column Alignment

Once the number of columns is known, the `UNION` clause becomes a powerful tool for extracting arbitrary data from the database. The `UNION` operator combines the results of two `SELECT` statements, provided they have the same number of columns

and compatible data types. By crafting a malicious `SELECT` statement that returns specific database information and `UNION` ing it with the original query, an attacker can force the database to display the desired data.

- **Leveraging an Empty Result Set:**

For the `UNION` attack to clearly display the injected data, it's often beneficial for the original `SELECT` query to return an empty set. This ensures that only the results from the attacker's injected `SELECT` statement are visible. The initial query used for `ORDER BY` (e.g., `WHERE username = 'test' AND password = 'any'`) is perfect for this, as it's designed to return no legitimate data.

- **Extracting Database Version (`@@version`):**

Knowing the database version can be critical for an attacker, as it helps identify known vulnerabilities specific to that version. Using the previously discovered column count (5 columns), an attacker can inject a `UNION SELECT` statement to retrieve the version information:

```
SELECT * FROM accounts WHERE username = 'test' AND password = 'any' UNION SELECT 1, 2,
```

In this query:

- * The `UNION` operator combines the empty result from the first `SELECT` with the result of the second `SELECT`.
- * `1, 2, 3, @@version, 5` are placeholders. `@@version` is a system variable in many SQL databases (like MySQL) that returns the database version.
- * The numbers `1, 2, 3, 5` are used to fill the other column slots, ensuring the column count matches. The `@@version` is strategically placed in the 4th column, and the output will display the version string in that position.

- **Extracting Current Database User (`user`):**

Similarly, an attacker can extract the current database user, which can provide insights into the privileges available to the compromised connection.

```
SELECT * FROM accounts WHERE username = 'test' UNION SELECT 1, 2, 3, user, 5
```

- Here, `user` is another system variable that returns the current database user (e.g., `root@localhost`).
- The output will display `root@localhost` in the 4th column, revealing the user account under which the database is operating. This information is invaluable for further privilege escalation or targeted attacks.

- **Next Steps: Enumerating Table Names:**

While not explicitly demonstrated in the provided content, the logical next step after identifying the column count, version, and user is to enumerate table names. This would typically involve querying information schema tables (e.g., `information_schema.tables` in MySQL) using similar `UNION` techniques to discover all tables within the database, paving the way for extracting data from them.

Key Takeaways

- [] Error-Based SQL Injection leverages database error messages to gain information about the database structure.
- [] The `ORDER BY` clause can be used to determine the exact number of columns in a table by incrementing the sort column number until an error occurs.
- [] An `ORDER BY` error (e.g., `ORDER BY 6` on a 5-column table) directly reveals the column count.
- [] The `UNION` operator is critical for injecting malicious `SELECT` statements to extract data.
- [] For `UNION` attacks, the injected `SELECT` statement must have the same number of columns as the original query.
- [] It is often advantageous for the initial `SELECT` query to return an empty set so that only the injected `UNION` results are displayed.
- [] System variables like `@@version` and `user` can be extracted using `UNION` to reveal database version and current user.
- [] Metasploitable 2 provides a safe environment for practicing and understanding SQL Injection techniques.

Conclusion

The techniques demonstrated in this lecture underscore a fundamental principle in cybersecurity: what appears to be a minor system error can, in the hands of a skilled attacker, become a powerful tool for reconnaissance and exploitation. Error-Based SQL Injection transforms database error messages from simple debugging aids into inadvertent data leaks, revealing critical structural and credential information. By understanding how `ORDER BY` can enumerate column counts and how `UNION` can then be leveraged to extract sensitive data like database versions and user accounts, we gain crucial insights into the attacker's mindset.

For students and professionals alike, this knowledge is a dual-edged sword. It equips ethical hackers with the methods to identify and report such vulnerabilities, thereby

strengthening system defenses. Simultaneously, it provides developers and database administrators with a clear understanding of the attack vectors they must guard against. Moving forward, the principles learned here—the meticulous observation of system responses and the creative manipulation of SQL syntax—form the bedrock for comprehending more advanced injection techniques and, more importantly, for building robust, error-resilient, and secure web applications that do not inadvertently broadcast their internal secrets.